

## Generating geospatial trajectories with incomplete data using graph inverse reinforcement learning

Zi Hen Lin , A. Yair Grinberger & Daniel Felsenstein

**To cite this article:** Zi Hen Lin , A. Yair Grinberger & Daniel Felsenstein (27 Apr 2026): Generating geospatial trajectories with incomplete data using graph inverse reinforcement learning, International Journal of Geographical Information Science, DOI: [10.1080/13658816.2026.2658065](https://doi.org/10.1080/13658816.2026.2658065)

**To link to this article:** <https://doi.org/10.1080/13658816.2026.2658065>



© 2026 The Author(s). Published by Informa UK Limited, trading as Taylor & Francis Group



[View supplementary material](#)



Published online: 27 Apr 2026.



[Submit your article to this journal](#)



Article views: 38



[View related articles](#)



[View Crossmark data](#)



RESEARCH ARTICLE



OPEN ACCESS



# Generating geospatial trajectories with incomplete data using graph inverse reinforcement learning

Zi Hen Lin<sup>a</sup>, A. Yair Grinberger<sup>b</sup> and Daniel Felsenstein<sup>b</sup>

<sup>a</sup>Department of Computer Science, Technical University of Munich, Germany; <sup>b</sup>Department of Geography, The Hebrew University of Jerusalem, Israel

## ABSTRACT

Geographic simulations of human systems often rely on trajectory generation to model agent behavior. This paper addresses trajectory generation under conditions of limited data. We demonstrate how deep learning can reproduce representative urban mobility patterns using two datasets for Jerusalem: a GPS-based travel survey and a GIS dataset of all buildings. To mitigate data mismatch between both datasets, we apply information propagation via multi-dimensional clustering. To exploit the underlying geospatial network structure characterized by transition matrices, we use graph neural networks (GNNs) as reward function approximators and compare them with linear method and multi-layer perceptron as baselines. For the baselines, transition probabilities dominate behavior, while GNNs learn from global data patterns or local neighborhood structures. Results show that GNN-derived rewards correlate more strongly with building floorspace, influencing transition probabilities. We test several initial and terminal state configurations to generate trajectories based on learned policies. In the absence of a universally accepted evaluation method, we assess trajectories using aggregated performance indicators rather than individual trajectory similarity. Findings indicate that GNN-based rewards can improve the realism of simulated trajectories, though further refinement is needed.

## ARTICLE HISTORY

Received 1 July 2025  
Accepted 6 April 2026

## KEYWORDS

Simulation; geospatial trajectories; data constraints; machine learning

## 1. Introduction and state of the art

Geographic simulations that include mobile entities such as city residents, workers or visitors, require the generation of trajectories in order to model the behavior of the entities populating the simulation. To resemble reality, simulation data needs to contain rich spatiotemporal properties of agent routines. Generating this highly granular data is fraught with challenges. The first is that spatial dependence is likely to exist

**CONTACT** A. Yair Grinberger [yair.grinberger@mail.huji.ac.il](mailto:yair.grinberger@mail.huji.ac.il)

Supplemental data for this article can be accessed online at <https://doi.org/10.1080/13658816.2026.2658065>.

© 2026 The Author(s). Published by Informa UK Limited, trading as Taylor & Francis Group

This is an Open Access article distributed under the terms of the Creative Commons Attribution-NonCommercial-NoDerivatives License (<http://creativecommons.org/licenses/by-nc-nd/4.0/>), which permits non-commercial re-use, distribution, and reproduction in any medium, provided the original work is properly cited, and is not altered, transformed, or built upon in any way. The terms on which this article has been published allow the posting of the Accepted Manuscript in a repository by the author(s) or with their consent.

between neighboring units such as buildings, neighborhoods and even cities. This results in behavioral spillovers over contiguous space as actions in one spatial unit influence those in a neighboring unit. A second challenge is temporal. The serial nature of time means that temporal dependence will generate autoregressive behavior. What happens at a given time is affected by behavior in the previous period. The availability of highly granular spatiotemporal socio-economic data is limited. One way of generating it is via voluntary participation but this is often costly and noisy (Siła-Nowicka *et al.* 2016, Goodchild *et al.* 2017, Zhang and Zhu 2018). Trajectory data collected in the past do not often share the coverage and resolution currently required. Consequently, trajectory generation becomes a natural alternative, especially for downstream geographic simulation involving modelling responses to policy interventions such as congestion pricing and autonomous vehicle planning in transportation (de Palma and Lindsey 2011, Kong *et al.* 2023), mobility constraints resulting from pandemic containment measures (Toger *et al.* 2021, Kwak *et al.* 2021, Lu and Giuliano 2023) or the provision of emergency shelter in the advent of a natural hazard (Chen and Zhan 2008, Dawson *et al.* 2011).

This paper deals with trajectory generation in geographic simulation with limited data availability as an input to the modeling process. We highlight the use of machine learning to generate empirical and representative trajectories under data-constrained settings. As befits a case study grounded in simulation where trajectory data is required for hypothetical scenarios, our approach stresses producing unseen and diverse trajectories, that is, generating new trajectories consistent with aggregate patterns. This stands in contrast to replicating observed trajectories with high accuracy where trajectory prediction emphasizes precision in relation to objective goals such as distance minimization or cost optimization. We illustrate the case of generating urban mobility patterns for agents in Jerusalem using two data sets. The first is a mobility data set derived from a unique GPS survey of travel habits in the city. The second is a GIS data set covering all buildings in the urban area.

To uncover the natural sequencing in mobility behavior, one would generally derive motivation observationally from the data. A standard approach is to use an end-to-end supervised learning paradigm, for instance, the sequence-to-sequence approach popular in the area of Natural Language Processing (Sutskever 2014, Vaswani *et al.* 2017). This approach directly assimilates a set of training sequences, generates new sequences in the form of autoregressive predictions and optimizes predictions against empirical data. However, in the context of geospatial trajectories, it is difficult to formulate the trajectory-generation problem under such a setting, due to the difficulty of evaluation and irregular sequence length, on top of potential privacy concerns. It also requires a large volume of trajectory data, which is often not available in order to achieve good performance. Moreover, using optimization to generate trajectories highly similar to the empirical trajectories defeats the original purpose of trajectory generation.

Reinforcement Learning (RL) is less demanding in terms of data volume. Due to constant interaction with the environment, the approach strikes a balance between data exploitation and exploration while uncovering entities' actions and the motivations driving them. Given knowledge about the set of possible states – the state space – and the rewards attached to each state, i.e. the reward structure, a mobile entity

(agent) can be trained to achieve its target based on partial observations about its environment. This is achieved by finding an optimal policy (the probability distribution of actions at a given state) which maximizes cumulative rewards with constant feedback (rewards) for each action. The true reward structure for real-world dynamics such as geospatial trajectories is difficult to obtain (Safarzadeh and Wang 2024). Therefore, in this paper, we use Inverse Reinforcement Learning (IRL) as a suitable learning paradigm to extract the mobility reward structure from empirical trajectories. This is assumed to embed optimal real-world policy. It quantifies the behavioral motivations (rewards) that explain an observed sequence of actions (data) and then generates decision rules (policies) based on this behavior by fine-tuning the presumed transition dynamics. In this way, both observed and unobserved decision rules can be extracted to shape an agent's behavior in a simulation.

A limitation on the use of RL is that agents may invariably only have a vague idea of the reward function whose optimization results in 'desirable' behavior (Ng and Russell, 1998). As a result, RL is not pertinent as there is no behavior to reinforce and the agent does not always have expert agents whose behavior it can imitate. In IRL, the expert agents' reward function is recovered and used to generate desirable behavior. To operationalize IRL, entropy optimization is often used. The Maximum Causal Entropy Inverse Reinforcement Learning (MaxEntIRL) algorithm popularized by Ziebart *et al.* (2008) uses the maximum entropy principle. This is based on the least-wrong approach under which the distribution of behavior that maximizes entropy with minimal constraints, offers the 'least wrong' solution. Lee *et al.* (2017) have used a similar approach to create Schelling-type spatial and social segregation.

A recent application of this approach in an overtly spatial context is Oda (2021). He presents a multi-agent behavioral IRL model. This is applied in the context of ride-hailing services and matching self-interested driver behavior with rider needs. Using a Markov Decision Process (MDP) formulation, optimal policy is computed and the model learns the driver's reward function. Validation of the approach is done using real taxi trajectory data from Yokahoma city from which parameters of the cost function are estimated. On this basis the algorithm learns the equilibrium policy. The equilibrium visitation count is then calculated by repeating the policy propagation step and comparing at each stage to the Mismatch Distance Ratio (MDR) generated baseline.

Our paper contributes to the use of IRL and extends its utilization for trajectory generation by developing a learning and trajectory generation framework. This framework generates single-day, home-based mobility tours, i.e. ordered sequences of visited locations that start and end at a home location, without time-of-day annotations, by learning rewards from individual mobility records and integrating them with spatial transition probabilities. This learning-based approach offers a complementary perspective to mechanistic models such as the Lévy flights, Continuous-Time Random Walk, and especially the Exploration-Preferential Return (EPR) model and its extensions (Song *et al.* 2010, Pappalardo *et al.* 2016, Cornacchia and Pappalardo 2021, Schneider *et al.* 2026). While our treatment of within-day temporal patterns is more abstract, we systematically incorporate environmental semantics such as building attributes, spatial connectivity, and socioeconomic features in order to provide interpretable insights into behavioral motivations through learned reward functions. The framework

prioritizes understanding the spatial drivers of mobility and enabling counterfactual analysis over temporal prediction, making it suited for urban planning and design applications rather than operational forecasting.

This framework addresses three main challenges characteristic of previous applications of the IRL approach. The first is providing a clear strategy for the common geospatial challenge where the collected trajectory data cover only a subset of the desired state space. This has not been emphasized in previous work e.g. Ziebart 2010, Oda 2021. Second, we use the Laplacian matrix of a graph to offer a systematic solution to the known MaxEntIRL convergence issue (Ziebart 2010, Barnes *et al.* 2023). Third, as our preliminary findings suggest that the linear reward function of MaxEntIRL hinders the learning of complex geospatial dynamics, we follow Wulfmeier *et al.* (2015) and incorporate graph-based deep learning to approximate non-linear reward functions along with an in-depth impact analysis. This approach has not been applied in previous MaxEntIRL-based work with the exception of Wang *et al.* (2023a,b), which does not belong to geospatial domain.

We continue by presenting the empirical data and outlining the methodological and technical issues at stake for learning rewards and generating trajectories with incomplete data (Section 2). Section 3 presents the empirical analysis and the solutions to the aforementioned issues and Section 4 discusses the implications of these findings. Section 5 concludes with some observations on the consequences of our approach for future advances in geographic simulation modeling.

## 2. The learning and trajectory generation framework

In this section, we introduce the empirical datasets and the main components for operationalizing the MaxEntIRL algorithm (Section 2.1), including the feature matrix, transition matrices and the initial and terminal states. We follow this by discussing the trajectory generation approach (Section 2.2). The general workflow for applying the MaxEnt IRL algorithm to the case of geospatial trajectories is depicted in Figure 1.

Viewing a trajectory as a sequential MDP (Wulfmeier *et al.* 2015, Ziebart 2010), IRL requires pre-defining four elements: initial and terminal trajectory states, the state transition probability matrix and the expected state visitation frequency (ESVF). The latter is embodied in the feature matrix which is considered the materialization of the optimal policy (see Section 2.1.2). Initially, the procedure computes an intermediate policy using the feature matrix-based rewards, the terminal states and the transition matrix. This supports computing the ESVF, comparing it to the expert's ESVF and finally updating the reward function's weights based on the differences. When the loss function converges, the rewards are multiplied by the transition matrix to generate a policy, based on which trajectories are generated.

### 2.1. The learning framework

#### 2.1.1. Training data

We use a processed GPS trajectory dataset derived from a Travel Habits Survey conducted by the Jerusalem Transport Masterplan Team in 2011. This contains 7,692



mobility-relevant cutoffs. This process is discussed in more detail in Lin *et al.* 2025. While this approach may be construed as biasing the results, our tests indicate robustness (see Section 3.5). Dividing the number of visits per feature and cluster by the number of distinct buildings visited for this feature-cluster combination, we get the ‘expected visitation count’. With this, we fuse spatial-mobility information from heterogeneous sources into one feature matrix.

In order to derive building features, we use an additional dataset which contains all buildings in Jerusalem provided by the Survey of Israel (SOI). SOI is the Israeli governmental agency that produces official mapping and geodata. This dataset contains 38,093 buildings for Jerusalem with basic information such as coordinates, height, floorspace, and land use code. Due to computation limitations and for visualization purposes, we use a subset of 19 statistical areas (i.e. census tracts) containing 1,014 buildings. This is extracted from the computed feature matrix, for IRL training and trajectory generations (Figure 2).

### 2.1.3. The transition matrix

To simulate mobility dynamics between buildings, we generate transition probabilities using a simple gravity score (SGS). The gravity formulation is the bedrock of spatial interaction models relating to the transition of people from origins to destinations (Oshan 2021):

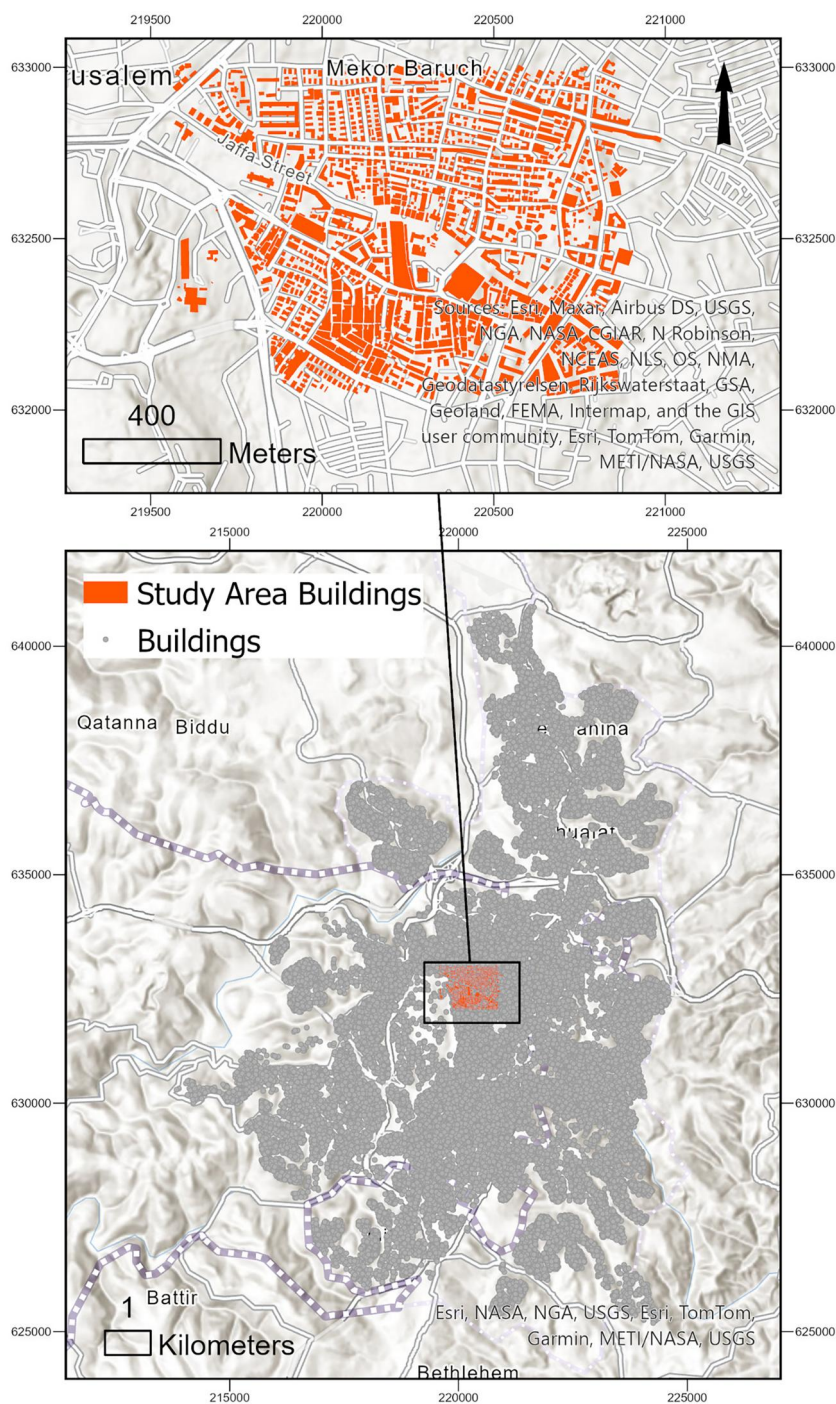
$$SGS_{ij} = \begin{cases} 0, i = j \\ \frac{V_i}{D_{ij}^2}, i \neq j \end{cases} \quad (1)$$

where,  $V$  is the volume or floorspace of a building and  $D$  is the distance between the two buildings. The  $SGS$  is then row-normalized to become a valid transition matrix. Below (Section 3.1) we discuss the impact of the transition matrix in terms of convergence and rewards. The transition matrix for MaxEntIRL is sparsified to 10% density to ease computation load.

### 2.1.4. Initial and terminal states

Since we have fundamentally altered the state space by representing buildings as a vector of cluster memberships (Section 2.1.2), we cannot follow the common practice of deriving initial and terminal states from the dataset (Kong *et al.* 2023). Instead, we address this as an issue of contextualization of trajectories, which can be characterized by a state space, initial states, terminal states, and dynamics. Our cluster-based state space removes the context from the mobility information while retaining the dynamics which are then propagated as expected visitation counts. For this application, we choose non-informative initial states and terminal states. We randomly assign all buildings in the study area as initial states and all residential and mixed-use buildings in the study area as terminal states. This approach assumes that we can propagate and refine the mobility dynamics and re-contextualize downstream trajectory generation by assigning specific initial states and terminal states (see Section 2.2).





**Figure 2.** Jerusalem study area: bottom: Jerusalem built-up area. top: buildings in the designated study area.



### 2.1.5. Nonlinearity of rewards

By defining the reward function as the product of the features weights and the feature matrix, MaxEntIRL assumes a linear relation between initial and final rewards (Ziebart 2010; Arora and Doshi 2021). We investigate the impact of this assumption by implementing four machine learning models as reward functions in PyTorch: Linear, Multi-Layer Perceptron (MLP), Graph Convolutional Network (GCN), and K-dimensional Graph Neural Network (GraphConv). The Linear reward model is a single feed-forward layer without activation function. The number of parameters is 11, matching the number of features in the input data. The MLP model is a basic form of neural network with a non-linear activation function. It is composed of three linear layers, interleaved with ReLU and BatchNorm layers. The hidden dimensions are 32. The number of parameters of MLP is 1569. MLP captures the global patterns and features in the data. Similar to MLP, the GCN and GraphConv algorithms have the same number of layers and the same activation and batch normalization functions. GCN and GraphConv are considered more advanced forms of reward models. They utilize the transition matrix as an extra input in addition to the feature matrix to characterize the underlying building network topology. In this way they can be considered as capturing local factors as each node gathers information from its neighbors. The number of parameters in each model depends on the number of layers and the number of channels in each layer. GCN replaces linear layers with GCN layers while GraphConv replaces them with GraphConv layers. These graph-based layers are supported by PyTorch Geometric. The number of parameters of GCN is the same as MLP, i.e. 1569, while that of GraphConv is 2945.

All models, aside from the linear model, incorporate non-linearity through non-linear activation functions. MLP acts as a control for non-linear reward functions to further differentiate the impact of geometrical inductive bias of graph-based models. GCN and GraphConv are both implemented to investigate the effects of different graph neural network (GNN) architectures.

### 2.1.6. Convergence and the Laplacian matrix

The convergence of computing the ESVF is a thorny issue in the implementation of the MaxEntIRL algorithm (Ziebart et al. 2008, Snoswell et al. 2020, Barnes et al. 2023). A naïve approach is computationally intractable and Ziebart's (2010) probabilistic approximation of the enumeration leads to convergence only under certain caveats on the matrix properties. Finding that the principal eigenvalue of the product of the probability distribution over actions at a given state  $\pi(\cdot)$  and the intermediate policy  $P(\cdot)$  dictates the convergence of this power iteration algorithm, we control the growth of the principal eigenvalue during the iteration by normalizing the product into its left Laplacian matrix (Figure 3). This technique successfully stabilizes the convergences and offers more insights about the choice of reward functions. Key insights relate to the fact that the principal eigenvalue of the Laplacian matrix is inversely proportional to the loss of ESVF matching and that graph-based deep learning models have higher initial principal eigenvalue than Linear and MLP, resulting in faster convergence.

**Algorithm: Expected state frequency calculation with Laplacian****Require:** MDP,  $\mathcal{M}_{\text{MDP}}$ , stochastic policy  $\pi(a_{x,y}|s_x)$ , and initial state distribution  $P_0(s_x)$ **Ensure:** state visitation frequencies,  $D_{s_x}$  under policy  $\pi(a_{x,y}|s_x)$ 

```

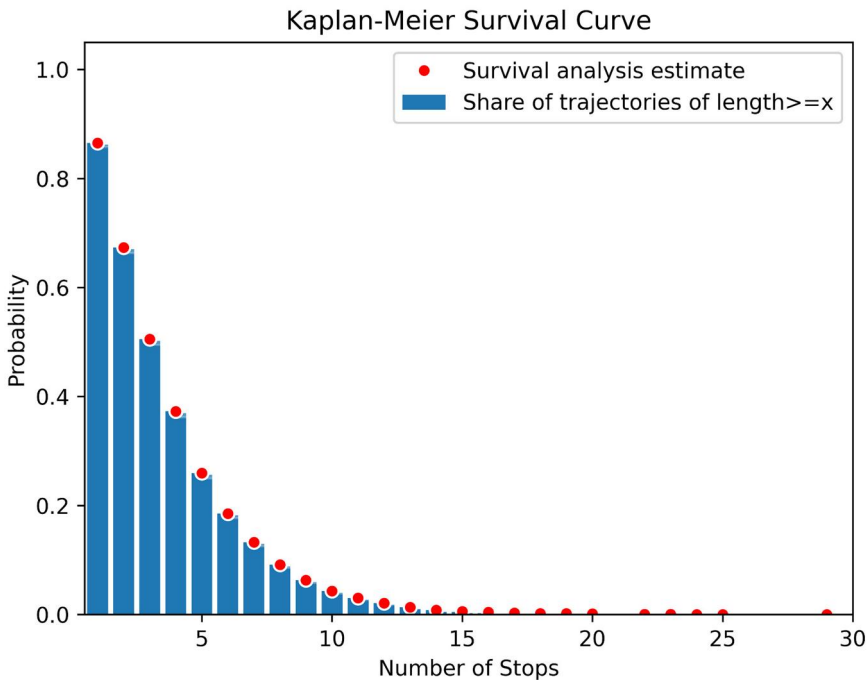
1 : for all states  $s_x \in \mathcal{M}_{\text{MDP}}$  do
2 :    $D_{s_x} \leftarrow 0$ 
3 : end for
4 : while not converged do
5 :   for all states  $s_x \in \mathcal{M}_{\text{MDP}}$  do
6 :      $D'_{s_x} \leftarrow P_0(s_x)$ 
7 :   end for
8 :   for all actions  $a_{x,y} \in \mathcal{M}_{\text{MDP}}$  do
9 :     for all states  $s_z \in \mathcal{M}_{\text{MDP}}$  reachable by  $a_{x,y}$  do
10 :       $D'_{s_z} \leftarrow D'_{s_z} + D_{s_x} \text{Laplacian}(\pi(a_{x,y}|s_x)P(s_z|a_{x,y}, s_x))$ 
11 :    end for
12 :   end for
13 :   for all states  $s_x \in \mathcal{M}_{\text{MDP}}$  do
14 :      $D_{s_x} \leftarrow D'_{s_x}$ 
15 :   end for
16 : end while

```

**Figure 3.** Pseudocode to stabilize the computation of expected state visitation frequency using the Laplacian matrix.**2.2. Trajectory generation**

After resolving the technical issues of MaxEntIRL implementation, we proceed to trajectory generation using the extracted rewards. A straightforward method for trajectory generation is a Markov-chain-like procedure in which, given an initial state, the next location in the trajectory is iteratively sampled from the transition (probability) matrix until a terminal state is reached. We assume that the residents go home every day, thus the initial and the terminal state are the home buildings of residents. Notice that initial and terminal states are different here from those discussed in [Section 2.1.4](#). Here we use specific states for the sake of generating a unique trajectory starting and ending at specific points, thereby recontextualizing the procedure, and not the general definitions for learning discussed above.

This back-to-home assumption generates trajectories visiting an unrealistic number of buildings before reaching the terminal state. Imposing a restriction that all buildings can only be visited once per resident only partially resolves the issue. To fully allay our concerns, we apply survival analysis to the trajectory dataset using the Kaplan-Maier estimator, which estimates the survival probability of each stop in the trajectory, as illustrated in [Figure 4](#). We consider survival probability as the discount factor for trajectory generation. The terminal probability is 1 subtracted by the survival probability. For each step we assign the terminal probability to the agent's home, and uniformly divide the survival probability for the remaining buildings. This set of probabilities is multiplied by the corresponding transition probability and rewards from IRL to obtain the final policy for sampling. With this, the generated trajectories have a long-tail distribution similar to that of the empirical data, ranging from 2 to 25 (we further evaluate the quality of the generated trajectories in [Section 3.4](#)). For the



**Figure 4.** Discount factor extracted from empirical trajectories dataset using survival analysis.

empirical dataset, all buildings outside the study area are excluded as well as trajectories with a length of 1. This results in 6567 empirical trajectories.

For the sake of illustration, we provide examples of generated trajectories in [Appendix 2 \(Figure A2\)](#). It is important to note here that these trajectories have no clear counterparts in the empirical dataset, as each trajectory is generated based on the updated transition matrix and not in relation to any specific empirical record. Hence, the assessment of the generated trajectories must occur by comparing the characteristics of the datasets, as we do in [Section 3.4](#).

### 3. Results

In this section we compare the linear and the three non-linear ML rewards models across a series of outcomes. Ultimately, we want to gauge the relative success of the different machine learning models in replicating the aggregate (i.e. average) empirical mobility patterns of city residents in the study area based on initial mobility and building data.

#### 3.1. Transition matrices

We use graph density as a metric for quantifying the number of edges in a graph compared to a fully-connected graph. Density levels express different levels of connectivity in a network. In our study area a density level of top 0.1%, yields around 700 connected components corresponding to the number of eigenvalues derived from

spectral graph analysis, with some buildings totally isolated. The top 1% density level results in 37 connected components. Beyond that, we observe only one connected graph. We experiment with a range of densities (0.1%, 1%, 10%, 30%, 50%, 80%) and in our context, different densities result in similar output. However, the higher the density, the heavier the computation load. The results of the learning procedure may be highly reliant on the choice of the transition matrix, with the learned policy ultimately producing a modified version of the input transition matrix. To assess procedure sensitivity to this choice, we implement another simple transition probability, the Inverse Euclidean Distance (IED), used as input instead of the SGS-based matrix, thus giving rise to different network structure:

$$D_{inverse} = 1 - D_{normalized} \quad (2)$$

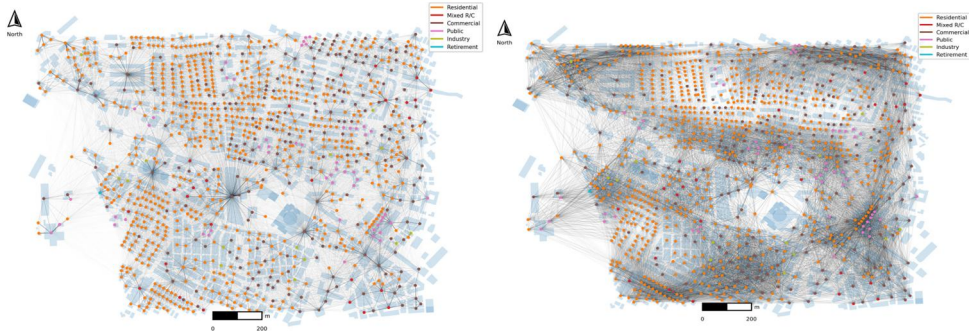
with  $D_{normalized}$  being the row-normalized version of  $D$ , the matrix of Euclidean distances between buildings.

In Figure 5 (right panel), the connectivity plot shows a few discrete clusters on the edge of the Jerusalem city center area rather than in the center itself. The transition matrix based on the SGS prioritizes larger buildings and shorter distances (Figure 5, left panel), with central buildings having more connections compared, as large floor-space volumes obviate relatively longer travel distances. The structure that emerges when SGS is used appears more sensible. It emphasizes existing activity centers thus supporting the use of this matrix.

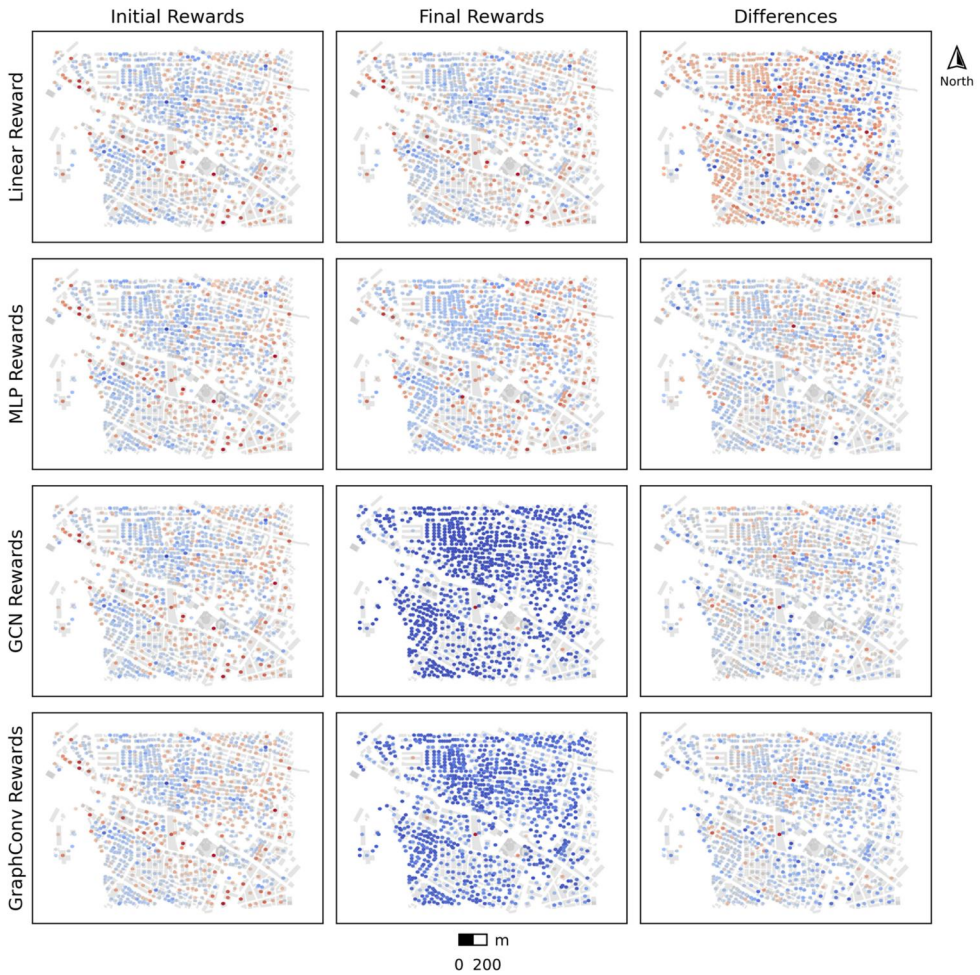
### 3.2. Rewards and transition matrices

Figure 6 presents the rewards upon initialization (left column), final rewards (middle column), and the differences between the two per building (right column) for linear and non-linear transformations based on the SGS matrix. It exposes minimal change for linear reward and the MLP. The graph-based models result in non-linear mappings, with high rewards concentrating in a small set of buildings, although the differences between the various neural network architectures are minimal.

This difference can be further illustrated with the correlation plots in Figure 7, where the rewards of GCN and GraphConv are highly correlated with building



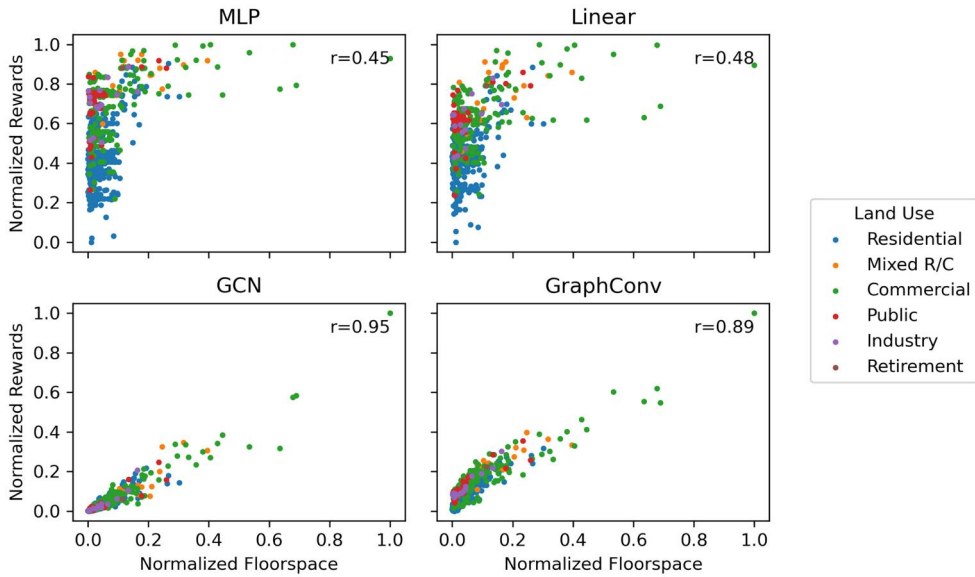
**Figure 5.** Connectivity plot based on the IED transition matrix (right panel) and the SGS (left panel). Nodes, representing buildings, are colored by land use, edge widths represent entry values. For illustration purposes, only the top 0.5% of the edges are shown.



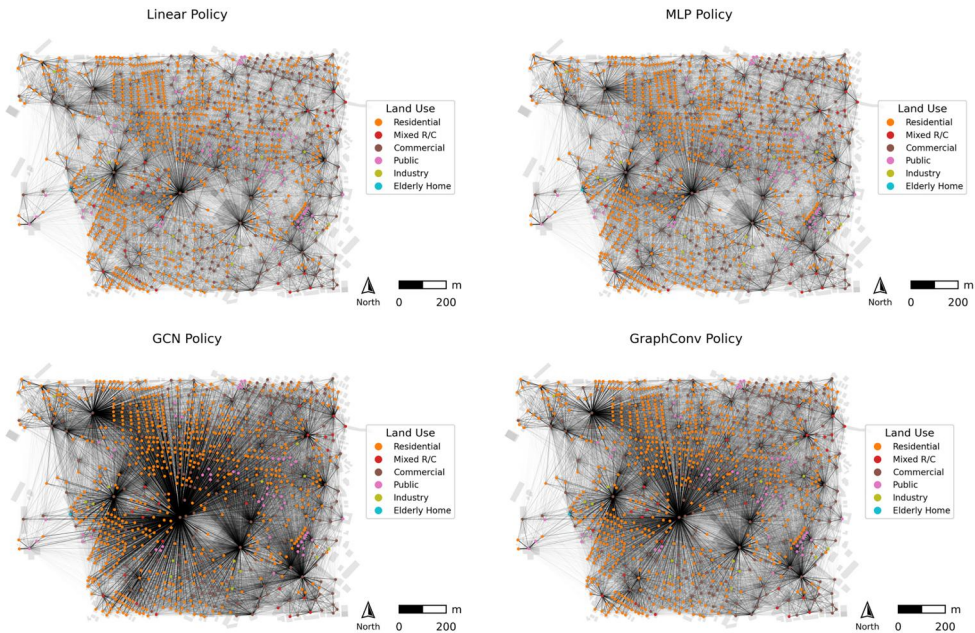
**Figure 6.** Spatial reward patterns for Linear, MLP, GCN, and GraphConv models using the Simple Gravity transition matrix. The warmer the color, the higher the standardized reward or the more positive the changes. The maximum rewards of GCN and GraphConv are manually rescaled to 0.8 for better visualization. We do not present exact values via legends in order to prevent clutter and the values are all standardized from 0 to 1.

floorspace. A different picture emerges from the Linear and MLP results where the correlations are less obvious. When observing the ultimate output of the learning procedure, that is, the policy as embodied in the updated transition matrix, the implications of this correlation emerge (Figure 8). While the linear and MLP models seem to densify the network without changing its basic structure compared to the original matrix (Figure 5 right panel), the geometric deep learning models create many more direct connections between small buildings and the buildings richest in floorspace. The results from Figures 7 and 8 suggest that graph-based deep learning models can identify the underlying dynamics in the given transition matrix and in this case, the central role of floorspace volume in determining attraction is highlighted. As buildings richest in floorspace are generally considered to be more attractive as they provide more





**Figure 7.** Correlation of floorspace and corresponding building rewards for four reward models using Simple Gravity Score transition matrix.

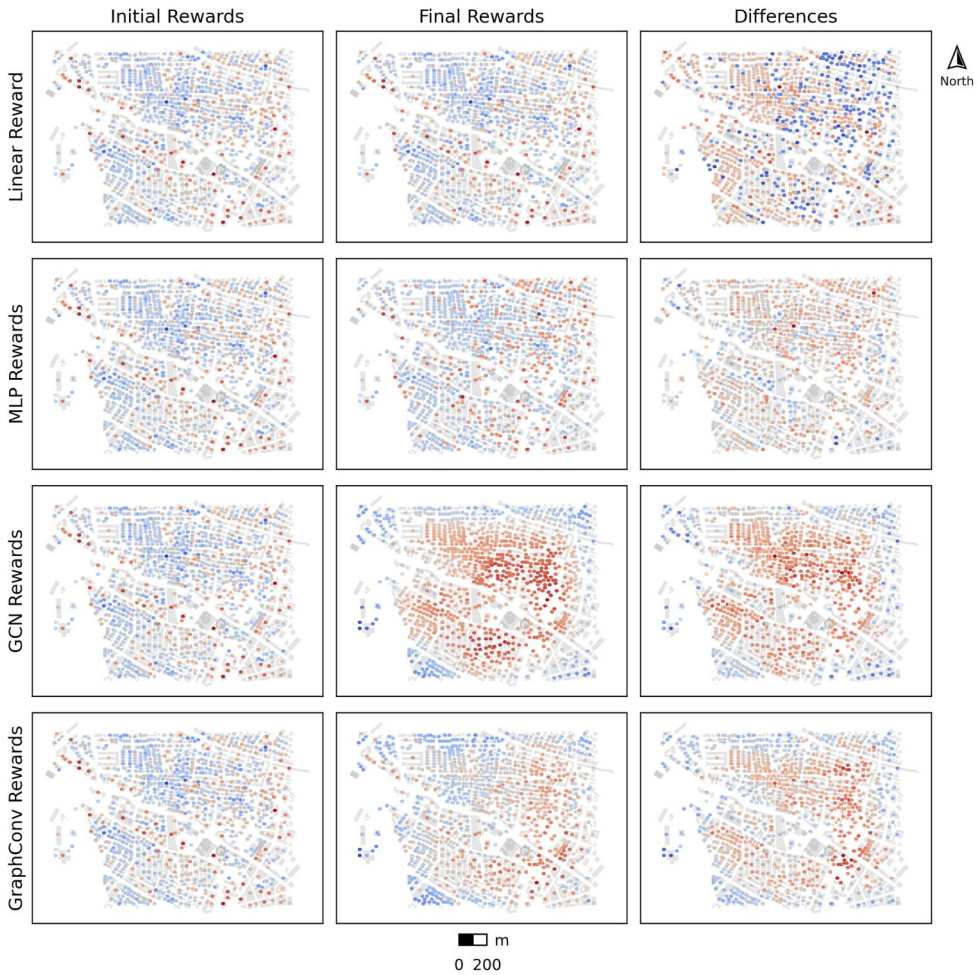


**Figure 8.** Resulting policies, i.e. updated transition matrices, for four reward models.

(commercial) opportunities, this provides a positive indication regarding the success of graph-based approach in producing realistic results.

In contrast, where the underlying transition matrix is IED, the GCN and GraphConv models seem to highlight the large number of buildings in the central area while relegating the peripheral buildings (Figure 9). It is interesting to note that the enclosed



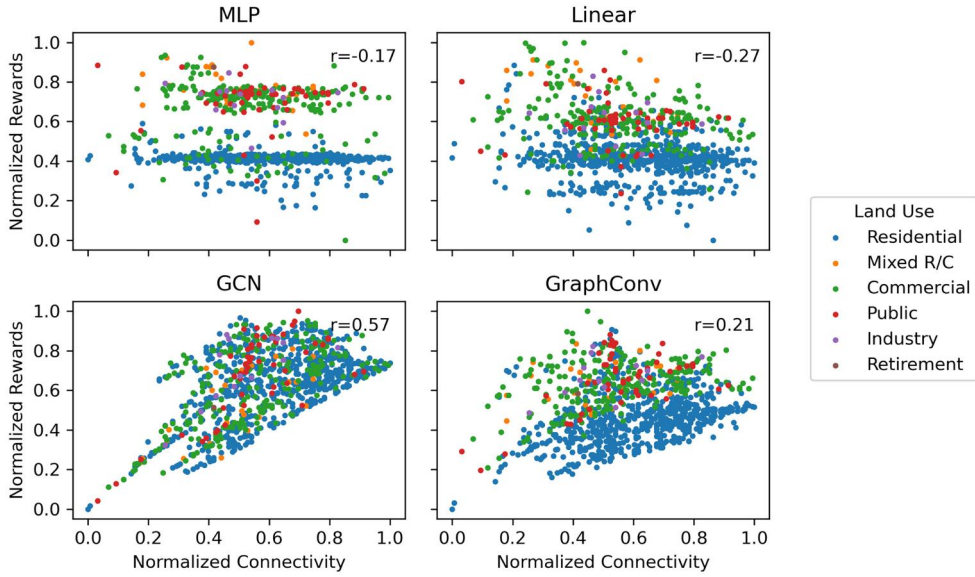


**Figure 9.** Spatial reward patterns for Linear, MLP, GCN, and GraphConv models using the IED transition matrix. The warmer the color, the higher the standardized reward or the more positive the changes.

market in the Jerusalem city center (Mahane Yehuda), the building with the most floorspace, gets a low reward compared to that received under the SGS scenario (Figure 10). Thus, GCN again demonstrates the ability to detect underlying transition matrix dynamics. The association between rewards and in-degree connectivity under the IED transition matrix ( $r=0.57$ ; Figure 10) reflects the IED prioritization of buildings with close proximity to neighbors, characteristic of the city center. As in the SGS case, MLP yields a similar final rewards distribution although the number of parameters is the same as that of GCN.

### 3.3. Trajectory generation, initial and terminal states

As discussed above (Section 2.2), trajectories are generated by sampling from the transition matrix until a building defined as a terminal state is selected. In the absence of a gold standard for evaluating trajectories, we compare across learning models,



**Figure 10.** Correlation of in-degree and corresponding building rewards for Linear, MLP, GCN, and GraphConv using Inverse Euclidean Distance transition matrix.

including the ‘untrained’ original matrix, using a series of performance metrics and comparing them to empirical trajectories. We compute common mobility metrics such as jump (i.e. segment distance) distribution and flow similarity, using the Common Part of Commuters (CPC) measure. For the first index, and also for total trajectory distance, number of buildings visited, and convex hull area, similarity to empirical trajectories is assessed using the Wasserstein distance between distributions. The CPC is by nature a comparative measure. Since a difference in extent exists between empirical and generated trajectories with the former covering the entire city and the latter only a subset of buildings, the CPC is computed based on flows between land use types (e.g. flows from residential to residential, residential to commercial, etc.). We specifically present the inverse CPC (i.e. 1-CPC, which has a maximum value of 1.0) so that all metrics in Table 1 follow the same direction and lower CPC is better than high CPC.

CPC results are sensitive to the underlying categorical composition (Lenormand *et al.* 2016), meaning that results depend on the initial composition of land use. Since these differ between the entire city and the study area, we define an alternative measure, the normalized flow metric (NFM). NFM is derived by dividing the observed relative flow (the share) for a given land use combination by the expected share of this combination under a random process (Eq. 3). To assess similarity between empirical and generated trajectories, we use the Root Mean Squared Error (RMSE) of NFM values across all origin-destination combinations.

$$NFM_{ij} = \frac{\%F_{ij}}{\%LU_i * \%LU_j} \quad (3)$$

where  $NFM_{ij}$  is the normalized flow metric between land use  $i$  and land use  $j$ ,  $\%F_{ij}$  is the share of trajectory segments starting in  $i$  and ending in  $j$  out of all trajectory

segments,  $\%LU_m$  is the share of buildings of land use  $m$  out of all buildings in the dataset.

We initially set the terminal state to be identical to the starting point, i.e. the agent's home. This reveals that GCN leads to the most similar distributions across most measures, closely followed by GraphConv (Table 1, configuration 1). The three measures deviating from this trend, trajectory length (number of stops), the CPC, and the NFM, show that the untrained model is superior. However, while the CPC shows a large difference between the GCN and the untrained model, the difference becomes minimal for the NFM. Interestingly, GraphConv performs much worse in terms of the NFM. Notice that both NFM and CPC relate to transition between land uses, meaning the geometric deep learning models perform better in recreating trajectory characteristics (length and area) than semantics (land use). When the terminal state is identical to the initial state this tends to generate trajectories which cover less area and distance and are based on shorter segments in comparison to empirical trajectories. This configuration also encounters edge cases where the agent cannot reach the initial state, in which case we reset the process and redraw the trajectory starting point. To resolve these issues, we explore two other configurations. In the first, all residential and mixed-use buildings are defined as terminal states. This resolves the edge cases issue improving performance in relation to trajectory length (Table 1, configuration 2). Differences in terms of CPC and NFM become minimal even if the untrained model remains the top performer for the CPC. However, this configuration performs worse in terms of several measures and hence we define a third configuration in which one of the residential/mixed-use buildings is randomly selected as the terminal state in an attempt to increase the probability that trajectories will cover more ground. This indeed improves the performance of some measures (Table 1, configuration 3), but the edge cases issue re-emerges. This requires us to redraw both the starting point and the terminal state when resetting the process and thus it is not surprising to see the NFM issue reemerging.

**Table 1.** Metrics for generated trajectories, for the four reward models and an untrained transition matrix, under three terminal state configurations.

| Configuration | Model     | Total<br>Trajectory<br>Distance<br>(log) | Mean<br>Segment<br>Distance<br>(log) | Trajectory<br>Length | Convex Hull<br>Area (log) | Inverse CPC | NFM (RMSE)   |
|---------------|-----------|------------------------------------------|--------------------------------------|----------------------|---------------------------|-------------|--------------|
| 1             | Untrained | 2.43                                     | 2.19                                 | <b>1.69</b>          | 4.39                      | <b>0.21</b> | <b>13.84</b> |
|               | Linear    | 2.36                                     | 2.12                                 | 1.80                 | 4.24                      | 0.28        | 17.04        |
|               | MLP       | 2.38                                     | 2.09                                 | 1.81                 | 4.18                      | 0.30        | 18.32        |
|               | GCN       | <b>1.98</b>                              | <b>1.74</b>                          | 2.47                 | <b>3.53</b>               | 0.44        | 15.91        |
|               | GraphConv | 2.13                                     | 1.88                                 | 2.09                 | 3.79                      | 0.38        | 27.65        |
| 2             | Untrained | 3.21                                     | 0.62                                 | 2.28                 | 5.81                      | <b>0.46</b> | 22.15        |
|               | Linear    | 3.10                                     | 0.88                                 | 2.28                 | 5.63                      | 0.46        | 22.01        |
|               | MLP       | 3.12                                     | 0.90                                 | 2.28                 | 5.68                      | 0.46        | <b>20.96</b> |
|               | GCN       | <b>2.72</b>                              | <b>0.56</b>                          | <b>2.27</b>          | <b>4.91</b>               | 0.49        | 22.44        |
|               | GraphConv | 2.89                                     | 0.69                                 | 2.28                 | 5.21                      | 0.47        | 21.43        |
| 3             | Untrained | 1.94                                     | 1.74                                 | 4.72                 | 3.19                      | 0.40        | <b>16.42</b> |
|               | Linear    | 1.86                                     | 1.67                                 | 4.72                 | 3.06                      | 0.40        | 26.09        |
|               | MLP       | 1.82                                     | 1.63                                 | <b>4.71</b>          | 2.97                      | <b>0.39</b> | 26.68        |
|               | GCN       | <b>1.50</b>                              | <b>1.31</b>                          | 4.83                 | <b>2.34</b>               | 0.44        | 21.33        |
|               | GraphConv | 1.61                                     | 1.41                                 | 4.85                 | 2.54                      | 0.41        | 29.90        |

### 3.4. City structure experiment

The experiments so far have assumed a constant city structure with the same number of buildings. In this section, we investigate the behavior of the proposed trajectory generation approach with moderate changes to the city structure using three hypothetical scenarios without re-training the reward models. This is motivated by the results above which show that while changing the configurations improves results, dissimilarities still exist between generated trajectories and empirical patterns. This may be explained by spatial incongruity between the training and implementation phases, with the first relating to the entire city and the other to a small case study area. Training the model using a large set of buildings tends to iron out the effect of distance, due to increasing the SGS denominator, thus increasing travel distances. Using a small subset of buildings, on the other hand, enhances distance effects and thus encourages shorter movement segments.

We test the validity of this explanation by reducing the number of buildings. This is done twice: first, we systematically remove buildings on the periphery of the area and then we repeat with random removal. This should decrease the SGS denominator, thereby increasing dissimilarities with the empirical patterns. In the first case, distance is expected to play an even greater role as the spatial extent of the study area is clipped. This is expected to lead to an even greater increase in dissimilarity in relation to the random removal scenario in which new formed gaps between buildings encourage traveling longer distances. Our third experiment takes a different approach, changing the urban structure by manipulating building floorspace volumes to inspect the inference response of reward models against new inputs, which could lead to many interesting applications in urban planning. Specifically, we take four peripheral commercial buildings of low to medium floorspace and increase their raw floorspace value to 10000 (i.e. amongst the top 10 largest floor space buildings). This means placing them in the category of the floorspace-rich with a floorspace feature value of 1.0. This experiment is expected to also increase travel distances, as peripheral locations should become more attractive.

For efficiency, we only present here results corresponding to one terminal state configuration (agent's home, corresponding configuration 1 in [Table 1](#)). This can be compared with results for the remaining configurations available in [Appendices 3–5](#), together with the respective supplementary connectivity plot of the test scenarios.

Our first test includes compressing the study area by excluding around 40% of the buildings (599 remaining buildings) located at the edge of the study area ([Figure 11](#)). Learned rewards for the chosen buildings remain unchanged to assess the robustness of the rewards against changes in city structure, and only the transition matrix is row-wise re-normalized. Interestingly, the results show similarity slightly dropping across all models and for all measures, except the CPC and the NFM. The general trend of GCN being the top performer across most measures is reiterated and even extends to the NFM, where the GCN performs the best and improved performance is also registered by GraphConv. These findings suggest that the exclusion of buildings by clipping the study area does not change the urban structure which is dictated by the buildings in the city center. The connectivity plots produced under this scenario ([Figure A3](#), [Appendix 3](#)) support this contention. This indicates that mobility dynamics are not



**Figure 11.** Buildings excluded (in grey) and used (in red) for the study area compression scenario (left) and the random removal scenario (middle). For the building editing scenario (right), the buildings in red are edited in terms of floorspace.

**Table 2.** Metrics for generated trajectories, for the four reward models and an untrained transition matrix, under three experiments.

| Experiment         | Model     | Total Trajectory Distance (log) | Mean Trajectory Segment Distance (log) | Trajectory Length (Buildings) | Convex Hull Area (log) | Inverse CPC (Building Type) | NFM (RMSE)   |
|--------------------|-----------|---------------------------------|----------------------------------------|-------------------------------|------------------------|-----------------------------|--------------|
| Reduced boundaries | Untrained | 2.67                            | 2.43                                   | <b>1.52</b>                   | 4.83                   | <b>0.18</b>                 | 18.96        |
|                    | Linear    | 2.52                            | 2.28                                   | 1.77                          | 4.55                   | 0.20                        | 16.73        |
|                    | MLP       | 2.51                            | 2.27                                   | 1.85                          | 4.53                   | 0.20                        | 16.10        |
|                    | GCN       | <b>1.90</b>                     | <b>1.65</b>                            | 3.44                          | <b>3.38</b>            | 0.34                        | <b>15.71</b> |
|                    | GraphConv | 2.15                            | 1.91                                   | 2.69                          | 3.85                   | 0.30                        | 17.26        |
| Random removal     | Untrained | 1.01                            | 0.78                                   | 1.71                          | 1.80                   | 0.24                        | 20.45        |
|                    | Linear    | <b>0.97</b>                     | <b>0.74</b>                            | 1.78                          | 1.75                   | 0.21                        | 18.39        |
|                    | MLP       | 1.02                            | 0.78                                   | 1.60                          | 1.78                   | 0.19                        | <b>17.40</b> |
|                    | GCN       | 0.98                            | 0.76                                   | 1.56                          | <b>1.73</b>            | <b>0.18</b>                 | 19.13        |
|                    | GraphConv | 1.03                            | 0.80                                   | <b>1.53</b>                   | 1.80                   | 0.19                        | 17.43        |
| Editing buildings  | Untrained | 2.43                            | 2.19                                   | <b>1.67</b>                   | 4.39                   | <b>0.22</b>                 | 15.24        |
|                    | Linear    | 2.32                            | 2.10                                   | 1.89                          | 4.18                   | 0.32                        | 20.68        |
|                    | MLP       | 2.34                            | 2.10                                   | 1.91                          | 4.22                   | 0.32                        | 19.59        |
|                    | GCN       | <b>1.91</b>                     | <b>1.67</b>                            | 2.51                          | <b>3.41</b>            | 0.45                        | <b>10.90</b> |
|                    | GraphConv | 2.03                            | 1.79                                   | 2.27                          | 3.62                   | 0.42                        | 13.01        |

constrained by the boundaries of the study area. Results for a selected area may be representative of a large-scale study area given sufficient similarity in terms of urban structure.

The second experiment complements the first by randomly removing roughly 30% of the buildings in the study area (709 remaining buildings), akin to the effect of a catastrophic event such as an earthquake (Figure 11). As before, we do not retrain the model but use a renormalized transition matrix based on previously learned rewards. The scenario shows increasing similarity in all measures (Table 2). A striking difference emerges in relation to the previous experiment: the four models now seem almost inseparable in terms of their performance. The connectivity plots (Figure A4, appendix 4) offer some explanation. The gaps emerging through the exclusion of buildings lead to increasing travel distances across the area, meaning that areas with abundant floor-space become more attractive. In other words, the exclusion of buildings does not change the basic structure of the area (i.e. no new centers emerge) but accessibility



and dynamics change. This change leads to the homogenization of model results, suggesting that the learned rewards based on previous transition probabilities cannot counteract the new dynamic, if a strong mismatch exists. While these results counter our expectations, by showing an increase in similarity, they actually support our general intuition that as travel distances increase, generated trajectories converge with their empirical counterparts.

The third experiment in which the floorspace volume of selected peripheral buildings is selectively increased in order to catapult them into the top 10 largest floorspace units, produces results very similar to those of the first experiment (Table 2) with the exception of results for the NFM. This means that even a moderate intervention that may simulate the establishment of new commercial centers, is not strong enough to offset the effects of the existing urban structure. The connectivity plots (Figure A5.1, Appendix 5) do show some change, especially for the GCN and GraphConv models, with the edited buildings becoming local focal points, yet these do not emerge as alternative anchors to the original focal points in the urban structure. This is related to the change in rewards for edited buildings, which is not uniform across the models. For all four models there is no direct correspondence of floorspace volume to rewards (Figure A5.2, Appendix 5).

It is worth noting that the various models display different reward dynamics in relation to the original (no experiment) rewards (Figure A5.3, Appendix 5). While this intervention results in a simple linear (or quasi-linear) offset in rewards under the linear and MLP models with little or no change for all other buildings, more complex patterns emerge for the GraphConv and GCN models. Under GraphConv, the rewards of all large buildings decline, except for the most attractive building (the covered market in the city center), suggesting a region-wide impact. The GCN model however leads to a less deterministic result with some buildings gaining in rewards and others losing. Interestingly, among the edited buildings, the building with the lowest original reward experiences the greatest increase. These all point to unique local dynamics and that the effect, in contrast with GraphConv results, is localized.

The emergent general trend points to an improvement in CPC and NFM values, especially under random removal and building editing for the CPC and for the geometric deep learning models (especially GraphConv) for the NFM. This finding suggests that a meaningful change to the urban structure increases similarity in terms of transition between land uses in relation to the city-wide empirical patterns. It supports the notion that the underlying structure of the case study area is different to the citywide structure. Such differences are manifested in incongruencies in terms of trajectory characteristics, e.g. total length and segment length.

### **3.5. Sensitivity tests – feature matrix definitions model parameters**

These experiments explore the sensitivity of the different models to changes in urban structure. Experiments 2 and 3 can also be interpreted as simulations of possible real-world scenarios. This raises the specter of other factors to which the results may be sensitive. We explore this by altering the thresholds used in the multi-dimensional

clustering procedure that generates the feature matrix and by changing the training models parameters.

For the first intervention, we run four tests: (a) reducing the threshold for the ‘high’ floorspace class to 1,000; (b) raising the thresholds for the ‘mid’ and ‘high’ floorspace classes to 500 and 1,000 respectively; (c) reducing all household income thresholds by 5,000NIS (New Israeli Shekel); (d) reducing the ‘low’, ‘medium’, and ‘high’ income thresholds to 15,000NIS, 18,500NIS, and 23,000NIS respectively. Tests (a) and (b) contrast with (c) and (d) by intervening with a physical rather social feature. Tests (a) and (c) both expand the higher value classes while test (b) expands the lower value classes, and test (d) shifts the distribution towards the extremes on both sides. We suffice here with presenting in the appendix the change in awards in relation to those produced by the original models (original feature matrix, no experiments; [Figure A6.1, Appendix 6](#)). These show that the geometric deep learning models produce stable results across configurations with correlation coefficients ranging from 0.84 to 0.86 for GraphConv and 0.92 to 0.95 for GCN. This contrasts with the sensitivity of the linear (0.62–0.65) and MLP (0.66–0.73) models. Test (a) seems to produce the most impact, mainly for the latter two models as the results across (b) to (d) are similar. This seems a sensible result given the central role of large buildings in the transition matrix.

The second intervention includes two additional tests: changing the number of neurons per layer from 32 to 256 (meaning increasing the number of parameters for MLP, GCN, and GraphConv to 69,889, 69,889, 138,241 respectively); changing the number of hidden layers from 1 to 3, keeping the number of neurons at 32 resulting in 3,089, 3089, and 7,233 parameters for the MLP, GCN, and GraphConv models respectively. The results ([Figure A6.2, Appendix 6](#)) again establish the GCN and GraphConv models as quite stable. When the network is wider (more parameters), these models converge faster but the training loss remains similar, indicating that they find the same optima. The main effect is a sharper separation between larger and smaller buildings. For the MLP model rewards decline for almost all buildings although training loss is lower. The GCN and GraphConv models reach the same optima when using a deeper network (more layers), producing almost the same results, in contrast to the MLP which lands on different sub-optima.

Together, the results of these sensitivity tests point to the utility of having geometrical identifiers through the transition matrix. They introduce an inductive bias which reduces the search space and guides the model towards the most parsimonious solution.

## 4. Discussion

The above results show that rewards generated by graph neural networks effectively capture underlying dynamics and have a stronger correlation with building floorspace than their linear counterparts. This sharpens the influence of the transition probabilities of the core buildings. Our experimentation with several settings of initial and terminal states to trajectory-generation based on learned policies, suggests sensitivity to network structure such as the edge-cases issue. Similar experimentation with manipulating the urban structure points to the inelasticity of the built environment.

Given the absence of any universally accepted benchmark that captures all aspects of trajectory realism, we propose assessing the aggregated properties of trajectories based on performance indicators instead of emphasizing individual similarity. The results suggest that using graph neural networks as a reward function generates improvements in the evaluation metrics. While these are not uniformly consistent across all metrics, for example improvements are larger for spatial structure (length and area) than for land use, overall the realism of trajectories is improved.

While our results do not entirely replicate empirical patterns, we contend that this is due to the nature of the case study rather than an artifact of our approach. The results of manipulating the case study by removing peripheral or randomly selected buildings seemingly undermines this hypothesis. However, exploring the dynamics behind the results shows that the greater similarity evinced under the random removal scenario is related to increasing travel distances derived from the gaps where buildings used to exist. This is coupled with a homogenization of results across models, pointing to the dominant impact of urban structure, that is expected to increase with travel distances.

Building on this, it seems that guiding trajectory generation using the transition matrix encourages short-distance travel. It is also clear that using an empirical dataset relating to the entire city as an accessibility space, while generating trajectories based on a subset of buildings, affects the results due to a mismatch in underlying mobility patterns. Our experiments in excluding buildings are a first step in examining this scaling issue. This effort warrants further attention in the future. The experiments suggest that generating trajectories for a larger area or compressing space by using a different distance matrix (e.g. travel duration) may improve the results.

The results however also support a surprising insight regarding model selection. While the use of graph neural networks reduces sensitivity to feature matrix definitions and leads to consistently more improvements in prediction accuracy due to inductive biases that specifically incorporate transition probabilities directly into the learning process, this advantage holds only when the transition probability matrix is correctly specified. Our results suggest that when the change in spatial structure exerts a strong enough impact, the utility of GNNs over simpler models is greatly reduced, unless the model is retrained using updated transition dynamics. Our results suggest that the latter may be more efficient for case studies with mild and generic mobility assumptions.

The spatial structure effect can be mitigated by controlling the density of the transition matrix. However, our current distance-based heuristics tend to prefer buildings with shorter distances unless outweighed by floorspace. The current approach of controlling density using quantiles is inherently linear and this encourages either global short distances or global long distances. In future work, a non-linear filter like a Gaussian filter to reduce noise or a filter that encourages a local maximum distance might be a better choice for reproducing empirical mobility patterns. Another promising future direction could be deriving a different transition heuristic, such as travel duration as mentioned above, or learning and optimizing the transition matrix using machine learning and numerical methods.

Aside from the transition matrix, other factors such as city structure, configuration of terminal states, discount factors and reward profile also contribute to the realism of the generated trajectories. This is reflected by the trajectory metrics and the connectivity plots. The order of these factors is arranged in terms of decreasing strength of influence. City structure has an influence in changing the transition matrix, and with certain changes, can dominate the impact of the configurations of terminal states. The discount factor is less dominant than the configurations of terminal states, otherwise the number of buildings in the generated trajectories will be the same across all configurations. The influence of rewards is largely localized, and best reflected in the connectivity plots after fine-tuning the mobility patterns on a microscopic scale.

## 5. Conclusions

This paper has aimed to fill the gap between theoretical advancement and practical settings in the context of trajectory generation by proposing a framework for generating realistic geospatial trajectories with incomplete empirical data using an IRL algorithm. To enhance our limited mobility data we use information propagation via multi-dimensional clustering to mitigate the data scarcity in the mobility dataset. We use graph neural networks as reward function approximators to exploit the underlying geospatial network structure characterized by transition matrices, resulting in better convergence and more interpretable building rewards. In this way we move from the classic machine learning approach to deep learning in which results are greatly determined by the feature matrix, to various non-linear alternatives that learn either from global patterns in the data (e.g. the MLP model) or from local features (such as the GCN model). The latter is of particular interest as it highlights the importance of learning from the environment as agents gather information from neighbors via the adjacency matrix and from graph topology. We illustrate this in a real-world setting of building networks rather than using a hypothetical grid-type world.

Our results show that rewards generated by graph neural networks effectively capture underlying dynamics and have a stronger correlation with building floorspace than their linear counterparts. This sharpens the influence of the transition probabilities of the core buildings. We also experiment with several settings of initial and terminal states to generate trajectories based on learned policies. Given the absence of a universally accepted benchmark framework for trajectory generation, we propose assessing the aggregated properties of trajectories based on performance indicators instead of emphasizing individual similarity. The results suggest that using graph neural networks as a reward function can indeed improve the realism of trajectories but with room for further improvement.

Future work in generating plausible mobility behavior will need to learn not just from graph topology, spatial structure and urban networks but also from the socio-economic structure of the city. Initial examination of our results at the building level suggests the distinct population groups in the city are related to different reward levels. For example, we find buildings associated with the Ultra-Orthodox community in Jerusalem consistently generate more rewards. This suggests that our approach to trajectory generation can ultimately lead to greater socio-economic refinement and

ultimately a more human-centric approach to urban simulation. This is as-yet uncharted territory that needs to be incorporated when generating meaningful patterns of urban spatial behavior.

## Acknowledgement

This research was supported by a Zelman Cowen Initiatives grant and by a grant from the Israeli Ministry of Innovation, Science, and Technology, through the Archimedes Center. We thank the three anonymous reviewers and the editor for their helpful comments. We would additionally like to acknowledge Shan Zhao for the fruitful discussions that helped shape this work.

## Author contributions

CRedit: **Zi Hen Lin**: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing; **A. Yair Grinberger**: Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing; **Daniel Felsenstein**: Conceptualization, Funding acquisition, Investigation, Methodology, Project administration, Resources, Supervision, Writing – original draft, Writing – review & editing.

## Declaration of AI assistance

During the preparation of this manuscript, we occasionally used ChatGPT (GPT-4o) to assist with language refinement and clarity of expression.

## Disclosure statement

No potential conflict of interest was reported by the author(s).

## Notes on contributors

**Zi Hen Lin** is a Doctoral Researcher in Informatics at the Technical University of Munich. His work focuses on machine learning for simulations, scientific computing, and infectious disease prediction, especially in relation to COVID-19 agent-based models and urban diffusion.

**A. Yair Grinberger** is a Senior Lecturer in the Department of Geography at the Hebrew University of Jerusalem. His fields of interest are GIScience, volunteered geographic information, mobility analysis, urban modeling, and the extraction of cultural knowledge from geodata.

**Daniel Felsenstein** is a Professor of Geography at the Hebrew University of Jerusalem. His research focuses on economic geography, regional science, spatial econometrics, simulation modeling, urban resilience, housing markets, and developing a spatial general equilibrium model for Israel.

## ORCID

Daniel Felsenstein  <http://orcid.org/0000-0003-0682-672X>



## Data and codes availability statement

The data and code that support the implementation reported in this research is available at <https://doi.org/10.6084/m9.figshare.29554394>.

## References

- Arora, S., and Doshi, P., 2021. A survey of inverse reinforcement learning: Challenges, methods and progress. *Artificial Intelligence*, 297, 103500.
- Barnes, M., et al., 2023. Massively scalable inverse reinforcement learning in Google Maps. *arXiv Preprint arXiv, 2305.11290* 2023. <https://doi.org/10.48550/arXiv.2305.11290>
- Chen, X., and Zhan, F.B., 2008. Agent-based modeling and simulation of urban evacuation: relative effectiveness of simultaneous and staged evacuation strategies. *Journal of the Operational Research Society*, 59 (1), 25–33.
- Cornacchia, G., and Pappalardo, L., 2021. A mechanistic data-driven approach to synthesize human mobility considering the spatial, temporal, and social dimensions together. *ISPRS International Journal of Geo-Information*, 10 (9), 599.
- Dawson, R.J., Peppe, R., and Wang, M., 2011. An agent-based model for risk-based flood incident management. *Natural Hazards*, 59 (1), 167–189.
- Goodchild, M.F., Aubrecht, C., and Bhaduri, B., 2017. New questions and a changing focus in advanced VGI research. *Transactions in GIS*, 21 (2), 189–190.
- Grinberger, A.Y., and Shoval, N., 2015. A temporal-contextual analysis of urban dynamics using location-based data. *International Journal of Geographical Information Science*, 29 (11), 1969–1987.
- Kong, X., et al., 2023. Mobility trajectory generation: a survey. *Artificial Intelligence Review*, 56 (S3), 3057–3098.
- Kwak, G.H., Ling, L., and Hui, P., 2021. Deep reinforcement learning approaches for global public health strategies for COVID-19 pandemic. *PLoS One*, 16 (5), e0251550.
- Lenormand, M., Bassolas, A., and Ramasco, J.J., 2016. Systematic comparison of trip distribution laws and models. *Journal of Transport Geography*, 51, 158–169.
- Lin, Z., Grinberger, A.Y., and Felsenstein, D., 2025. Simulating Covid-19 Contagion Patterns using a Machine Learning-Augmented Agent Based Model, in Broitman D, Kopczewska K, and Czamanski D (eds) *Handbook on Big Data, Artificial Intelligence and Cities* (pp. 327–348), Edward Elgar, Cheltenham, UK.
- Lu, Y., and Giuliano, G., 2023. Understanding mobility change in response to COVID-19: A Los Angeles case study. *Travel Behaviour & Society*, 31, 189–201. <https://doi.org/10.1016/j.tbs.2022.11.011>
- Ng, A.Y., 2000. Algorithms for Inverse Reinforcement Learning. In *Proceedings of the 17th International Conference on Machine Learning*, 663–670.
- Oda, T., 2021. Equilibrium inverse reinforcement learning for ride-hailing vehicle network. In *Proceedings of the Web Conference 2021*, pages 2281–2290.
- Oshan, T.M., 2021. The spatial structure debate in spatial interaction modeling: 50 years on. *Progress in Human Geography*, 45 (5), 925–950.
- de Palma, A., and Lindsey, R., 2011. Traffic Congestion Pricing Methodologies and technologies. *Transportation Research Part C: Emerging Technologies*, 19 (6), 1377–1399.
- Pappalardo, L., Rinzivillo, S., and Simini, F., 2016. Human mobility modelling: exploration and preferential return meet the gravity model. *Procedia Computer Science*, 83, 934–939.
- Safarzadeh, R., and Wang, X., 2024. Map matching on low sampling rate trajectories through deep inverse reinforcement learning and multi-intention modeling. *International Journal of Geographical Information Science*, 38 (12), 2648–2683.
- Schneider, M., et al., 2026. Generating semantically enriched mobility data from travel diaries. In Chrysanthos P K, Nøravåg K, Stefanidis K, and Zhang Z (eds) *Advances in Databases and Information Systems* (pp. 181–197), Springer, Cham.

- Siła-Nowicka, K., *et al.*, 2016. Analysis of human mobility patterns from GPS trajectories and contextual information. *International Journal of Geographical Information Science*, 30 (5), 881–906.
- Song, C., *et al.*, 2010. Modeling the scaling properties of human mobility. *Nature Physics*, 6 (10), 818–823.
- Snowwell, A.J., Singh, S.P.N., and Ye, N., 2020. Revisiting maximum entropy inverse reinforcement learning: new perspectives and algorithms. In *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 241–249.
- Sutskever, I., 2014. Sequence to Sequence Learning with Neural Networks. *arXiv Preprint arXiv*, 1409–3215.
- Toger, M., *et al.*, 2021. Mobility during the COVID-19 pandemic: a data-driven time-geographic analysis of health-induced mobility changes. *Sustainability*, 13 (7), 4027.
- Vaswani, A., *et al.*, 2017. Attention is all you need. In: I. Guyon, eds. *Advances in neural information processing systems*. Vol. 30. Curran Associates, Inc. [https://proceedings.neurips.cc/paper\\_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf)
- Wang, G., *et al.*, 2023a. Inverse reinforcement learning with graph neural networks for full-dimensional task offloading in edge computing. *IEEE Transactions on Mobile Computing*, 23 (6), 6490–6507.
- Wang, G., *et al.*, 2023b. Inverse reinforcement learning with graph neural networks for IoT resource allocation. In *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5. IEEE.
- Wulfmeier, M., Ondruska, P., and Posner, I., 2015. Maximum entropy deep inverse reinforcement learning. *arXiv Preprint, arXiv:1507.04888*. <https://doi.org/10.48550/arXiv.1507.04888>
- Zhang, G., and Zhu, A.-X., 2018. The Representativeness and Spatial Bias of Volunteered Geographic Information: A Review. *Annals of GIS*, 24 (3), 151–162.
- Ziebart, B.D., *et al.*, 2008. Maximum Entropy Inverse Reinforcement Learning, in *Proceedings of The Twenty-third AAAI Conference on Artificial Intelligence*, 1433–1438
- Ziebart, B.D., 2010. *Modeling interaction via the principle of maximum causal entropy*. PhD thesis, Carnegie Mellon University.